

INTRODUCTION

Communication between various components, including processors, memory, and peripherals, is crucial to the functionality and proper operation of today's embedded systems. Several communication protocols are utilised to properly manage this communication. The Advanced High-performance Bus, or AHB, is one of these protocols. It is intended to provide quick and efficient data transfer between different system devices.

Verification is an essential step in ensuring that certain communication protocols operate as intended. Verification is the process of confirming that the system operates as anticipated and satisfies the necessary requirements and specifications. Proper verification becomes even more important as designs become more complicated. **Verification IP (VIP)** is one of the finest techniques to make sure a protocol is operating correctly.

The creation and development of a Verification IP tailored to the AHB protocol is the main goal of this project. In order to ensure that AHB-based systems operate as intended and adhere to the AHB protocol, the **Verification IP** developed for this project is intended to be a tool that automates testing and verification.

LITERATURE REVIEW

Verification IP

Verification IP (VIP) is a tool used in the verification process. It is used to verify the functionality of a particular design under test. This is done by ensuring that the design that has been provided for testing follows the specified protocol. Different aspects of protocol, including timing, signaling and data transactions are verified. The verification of the system is done using the **Universal Verification Methodology (UVM)**, in which reusable and scalable testbenches are created. The hardware descriptive language, **System Verilog**, is used for verification. The UVM library used for verification is built on top of System Verilog, which provides a rich library of components to build testbenches. Using UVM along with SystemVerilog helps to verify protocols like AHB efficiently and ensures that the design meets all required specifications.

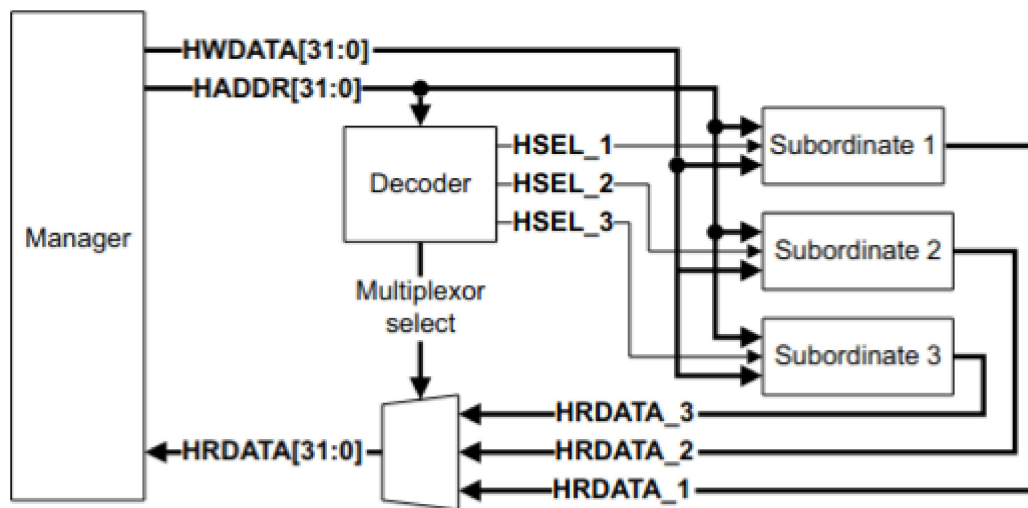
AHB Protocol

The **AMBA AHB (Advanced High-performance Bus)** is a bus interface created for high-performance systems that can be synthesized. It defines how different components, like Managers, interconnects, and Subordinates, communicate with each other. AHB is designed to meet the needs of fast systems, offering features like burst transfers, single-clock-edge operation, non-tristate implementation, and adjustable data and address bus widths. These features make it suitable for systems that work at high clock speeds.

In a standard AHB configuration, there is usually one **Manager** and several **Subordinates**. The **interconnect** logic is responsible for address decoding and data multiplexing. The address decoder checks the Manager's address during the address phase and selects the appropriate Subordinate for the data

phase. After that, the multiplexor sends the data from the chosen Subordinate back to the Manager.

AHB also supports systems with **multiple Managers**, where an interconnect component manages the arbitration and routing of signals from different Managers to the correct Subordinates. This setup allows for more flexibility and scalability in designing high-performance systems.

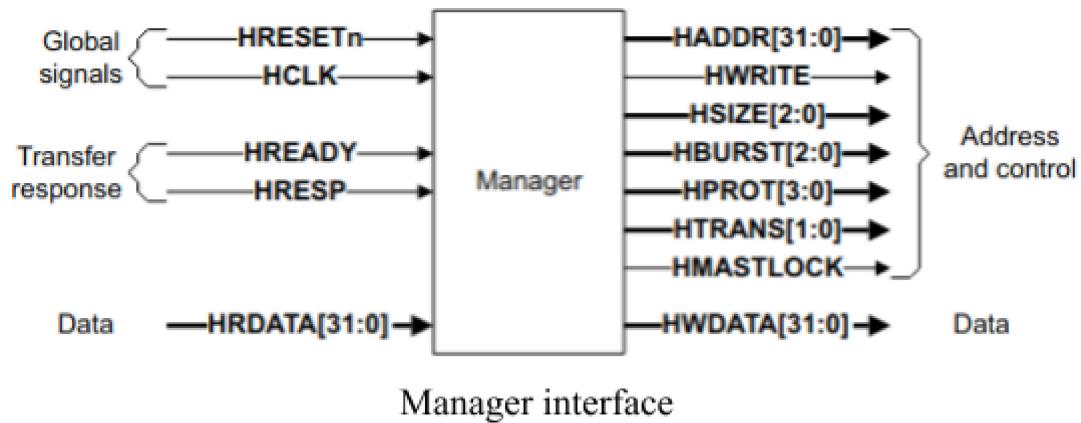


AHB block diagram

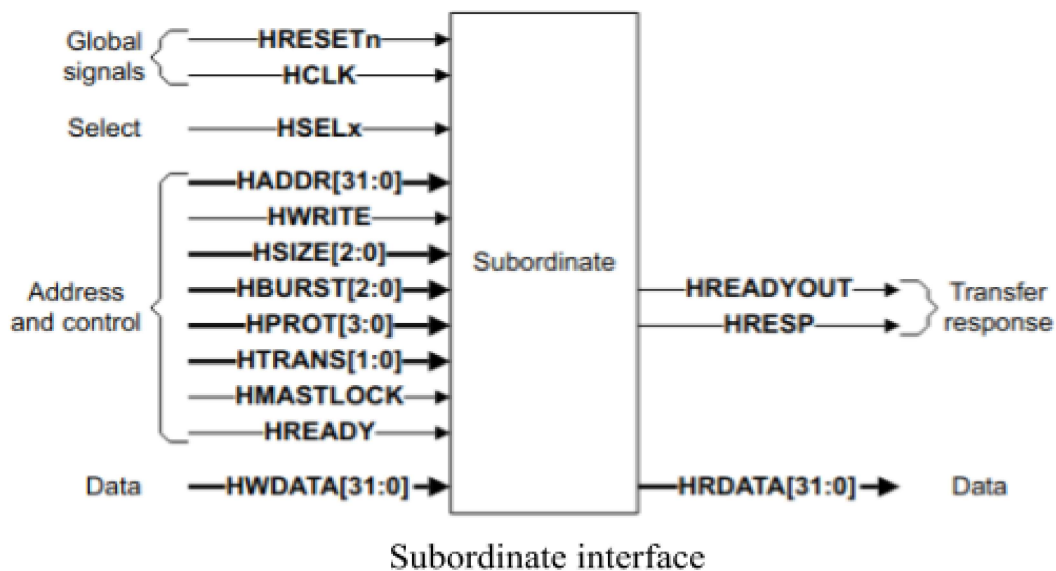
The main component types of an AHB system are described in:

- Manager
- Subordinate
- Interconnect

Manager : A Manager provides address and control information to initiate read and write operations.



Subordinate : A **Subordinate** responds to the transfers that are started by the **Managers** in the system. It uses the **HSELx** select signal from the decoder to decide when it should respond to a bus transfer. After that, the Subordinate sends signals back to the Manager to indicate whether the transfer is completed or if it needs to be extended. It also informs the Manager about the success or failure of the transfer.



Interconnect : An **interconnect component** connects the **Managers** and **Subordinates** in the system. In a system with just one Manager, only a

Decoder and **Multiplexor** are needed. However, for a system with multiple Managers, an interconnect is necessary to manage arbitration and route signals from the different Managers to the correct Subordinates. This routing involves address, control, and write data signals. The **Decoder** is responsible for decoding the address of each transfer and sending a select signal to the Subordinate involved, as well as controlling the multiplexor. In systems with two or more Subordinates, a single centralized decoder is required. The **Multiplexor** is used to route the read data bus and response signals from the Subordinates to the Manager. The decoder controls this multiplexor, and similarly to the decoder, a single centralized multiplexor is required when there are two or more Subordinates in the system.

Operation : The **Manager** initiates a transfer by sending the address and control signals, which contain information about the address, direction, transfer width, and whether the transfer is part of a burst. Transfers can be single, incrementing bursts that do not wrap at address boundaries, or wrapping bursts that wrap at specific address boundaries. The **write data bus** is used to transfer data from the Manager to a Subordinate, while the **read data bus** moves data from the Subordinate to the Manager.

Each transfer is made up of two phases. The first is the **address phase**, which consists of one address and control cycle. The second is the **data phase**, which involves one or more cycles for data transfer. The Subordinate cannot request an extension of the address phase, so it must be able to sample the address during this time. However, the Subordinate can ask the Manager to extend the data phase by using the **HREADY** signal. When **HREADY** is LOW, wait states are inserted into the transfer, allowing the Subordinate additional time to provide or sample the data. Finally, the Subordinate uses the **HRESP** signal to indicate whether the transfer was successful or failed.

Signal Descriptions

Global Signals :

HCLK: The bus clock is generated by the clock source and is used to time all bus transfers, with signal timings based on the rising edge of **HCLK**.

HRESETn: The reset signal, coming from the reset controller, is active LOW and resets both the system and the bus when activated.

Manager Signals :

HADDR: The byte address of the transfer, with width defined by **ADDR_WIDTH** (recommended between 10 and 64 bits), which specifies the address for the current transaction.

HBURST: Indicates the number of transfers in a burst and how the address increments, with a width of **HBURST_WIDTH** (must be 0 or 3), used to manage burst operations.

HMASTLOCK: A 1-bit signal that indicates the current transfer is part of a locked sequence, and it has the same timing as the address and control signals.

HPROT: A protection control signal with width **HPROT_WIDTH**, which provides information about the type of access, and its width can be 0, 4, or 7 depending on the **Extended_Memory_Types** property.

HSIZE: A 3-bit signal that indicates the size of the transfer, determining the amount of data to be transferred.

HNONSEC: A 1-bit signal that indicates whether the transfer is non-secure or secure, supported if the **AHB5 Secure_Transfers** property is enabled.

HEXCL: A 1-bit signal that shows if the transfer is part of an exclusive access sequence, supported if the **AHB5 Exclusive_Transfers** property is enabled.

HMASTER: A signal generated by the Manager, which identifies the Manager in case of multiple exclusive-capable threads, and is modified by the interconnect to uniquely identify each Manager; the width of this signal is defined by **HMASTER_WIDTH** (recommended between 0 and 8).

Subordinate Signals :

HRDATA: During read operations, the read data bus transfers data from the selected Subordinate to the multiplexor, which then passes it to the Manager, with a **DATA_WIDTH** that can range from 8 to 1024 bits, though values outside the 32-256 bit range are not recommended.

HREADYOUT: A 1-bit signal that indicates when a transfer has completed on the bus; it can be driven LOW to extend the transfer.

HRESP: A 1-bit signal that provides additional information about the transfer status; when LOW, it indicates the transfer is successful (OKAY), and when HIGH, it signals an error.

HEXOKAY: A 1-bit signal used to indicate the success or failure of an Exclusive Transfer, supported if the **AHB5 Exclusive_Transfers** property is enabled.

Decoder Signals :

HSELx: Each Subordinate has a unique select signal (**HSELx**) that indicates when a transfer is directed to it; the Subordinate must check **HREADY** to ensure the previous transfer is complete before responding, and for non-IDLE transfers, **HSELx** must be asserted in the same cycle as the address and control signals.

Multiplexor Signals :

HRDATA: The read data bus, selected by the decoder, carries the data from the Subordinate to the Manager during read operations.

HREADY: A 1-bit signal that, when HIGH, informs the Manager and Subordinates that the previous transfer has been completed.

HRESP: A 1-bit transfer response signal, selected by the decoder, that indicates the status of the transfer.

HEXOKAY: A 1-bit signal, selected by the decoder, that indicates whether the Exclusive Transfer was successful.

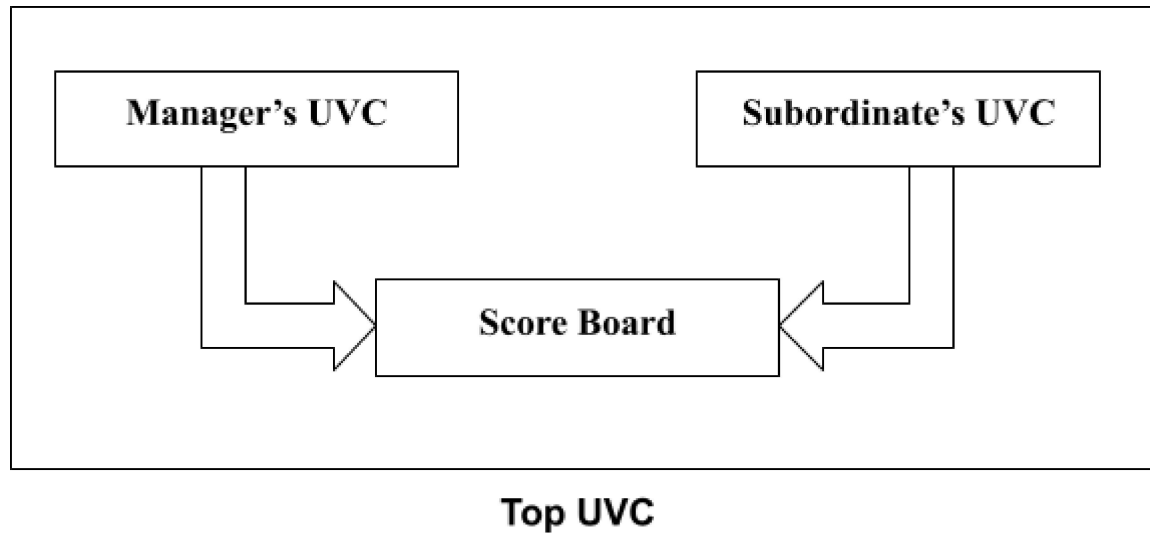
METHODOLOGY AND RESULTS

In this project, a Verification IP (VIP) for the AHB protocol is designed without a Device Under Test (DUT). The VIP is implemented using the Universal Verification Methodology (UVM) and SystemVerilog. The methodology followed in designing the VIP involves the creation of multiple Verification Components (UVCs), which are used to verify the functionality of the AHB protocol.

The first step is the development of a UVC for the Manager side of the protocol. This UVC is responsible for checking the interface on the Manager side and ensuring that it adheres to the rules of the AHB Manager. SystemVerilog Assertions (SVAs) are used to verify that the Manager follows all the required specifications.

Similarly, a second UVC is developed for the Subordinate side of the protocol. This UVC checks the interface on the Subordinate side and verifies that it complies with the rules of the AHB Subordinate. Again, SystemVerilog Assertions are utilized to confirm the proper functionality of the Subordinate.

Finally, a main UVC is designed, which includes both the Manager's UVC and the Subordinate's UVC. In addition to these, a Scoreboard is incorporated into the main UVC. The Scoreboard collects data from both the Manager and Subordinate UVCs and compares the information to determine whether data transfer is successful. This process ensures that the AHB protocol's functionality is thoroughly verified.



Manager's UVC

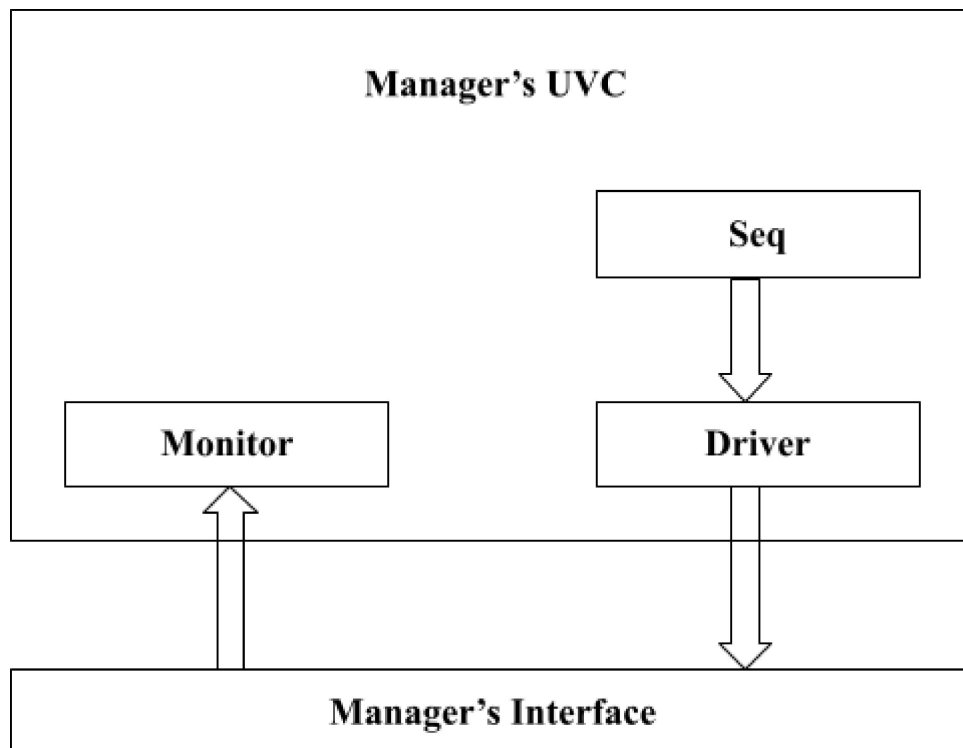
The Manager's UVC is one of the key components used in verifying the Manager side of the AHB protocol. It consists of some important components: the Sequencer, Driver, and Monitor. These components work together to verify the functionality of the Manager's interface.

The Sequencer is connected to the Driver and provides sequences to the Driver. These sequences are used to generate the transactions that the Manager will perform. The Driver is connected to the Manager's interface and drives these sequences onto the interface, simulating the behavior of the Manager.

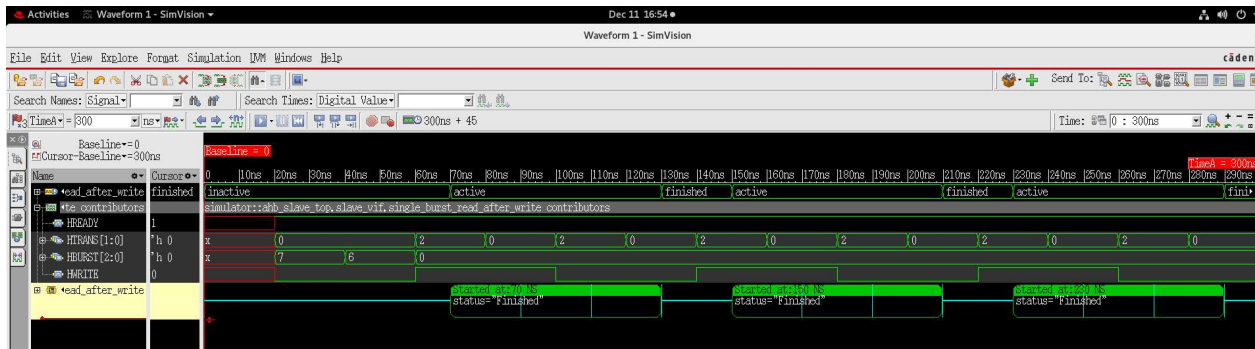
The Monitor is also connected to the Manager's interface. It receives the sequences from the interface and monitors the signals on the Manager side.

The verification of the Manager's side of the AHB interface is done by monitoring the signals through the 'Manager's Interface'. This process is accomplished using SystemVerilog Assertions in the interface. These assertions check whether the Manager's side of the interface follows the AHB protocol rules. In addition, coverage of the Manager's signals is also implemented in the interface.

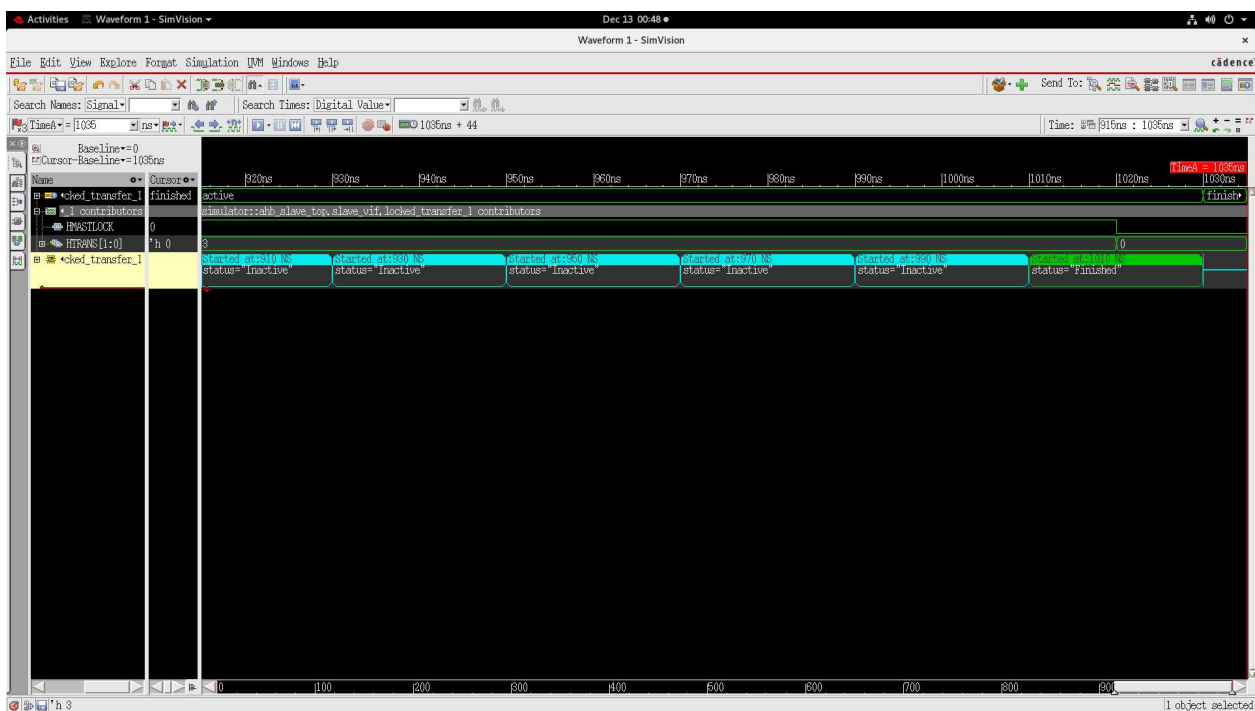
The assertions in the Manager's interface verify different types of transactions. These include Single Burst transactions, Increment transactions of types INCR4, INCR8, and INCR16, as well as Wrapping transactions such as WRAP4, WRAP8, and WRAP16. The assertions also check Infinite Burst type transactions. All of these checks ensure that the Manager's interface behaves correctly according to the AHB protocol.



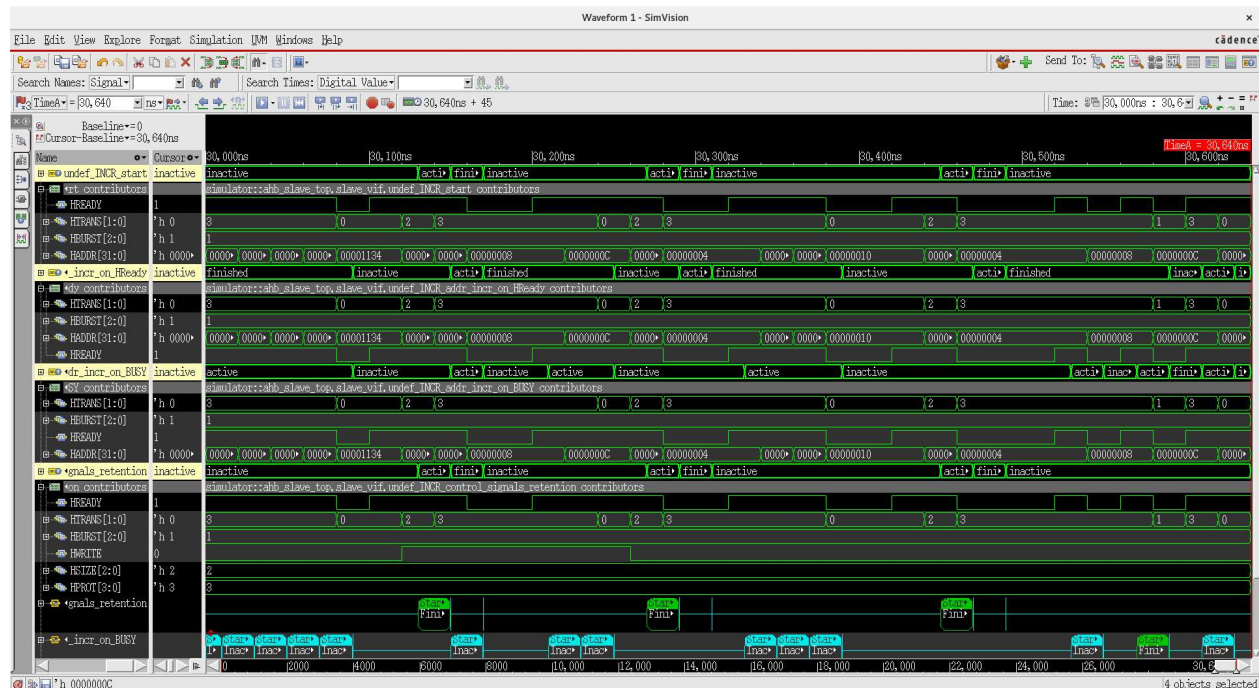
Below Snaps of various assertions of Manager's UVC are given. Firstly, the simple read after write assertion is :



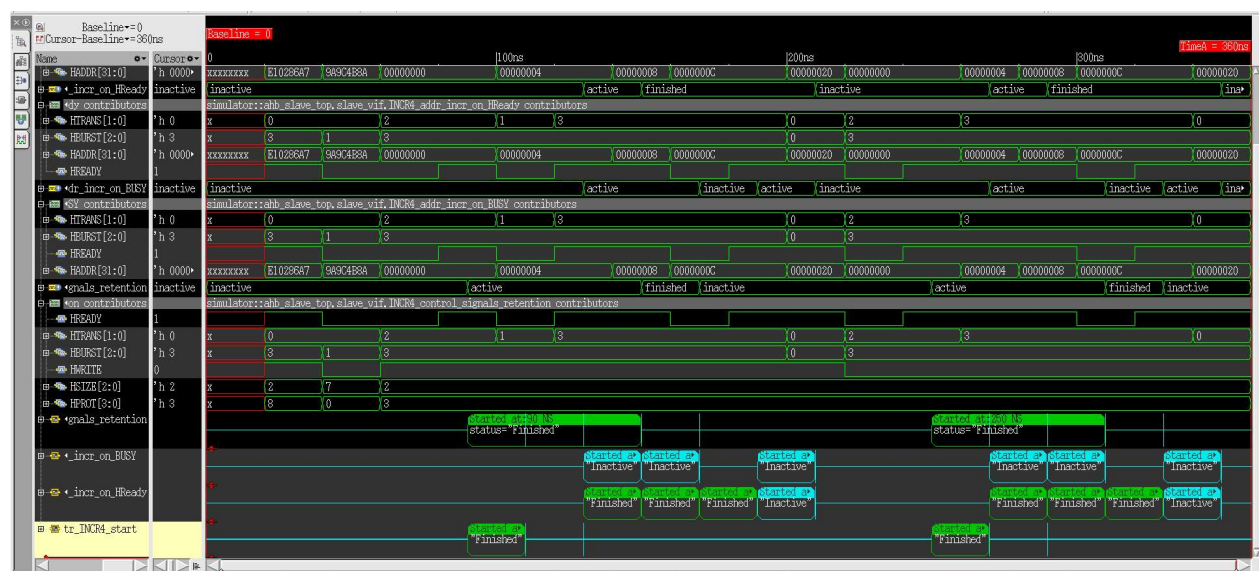
The snap of LOCK transfer assertion is :



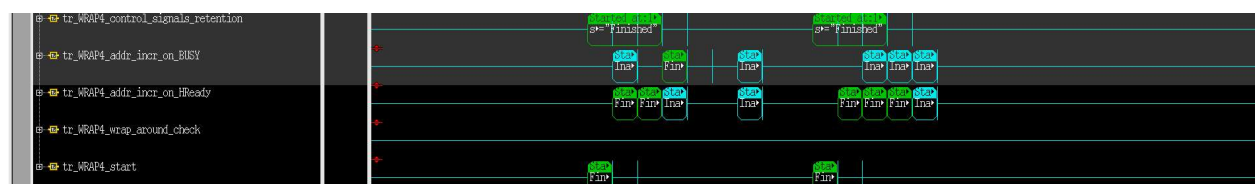
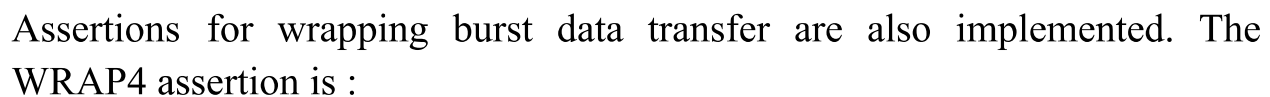
Similarly, assertions of undefined increment are :



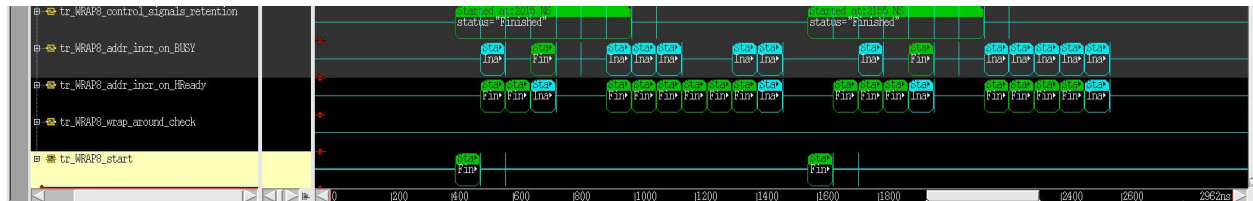
The screen shot of INC4 read after write assertion is attached :



The screenshot of INC8 read after write assertion is :



Also, the WRAP8 assertion is :



Similarly, WRAP16 assertion is :



Subordinate's UVC

The Subordinate's UVC is another important part of verifying the AHB protocol. Similar to the Manager's UVC, the Subordinate's UVC also consists of some key components: Sequencer, Driver, and Monitor. These components work together to check the functionality of the Subordinate's interface.

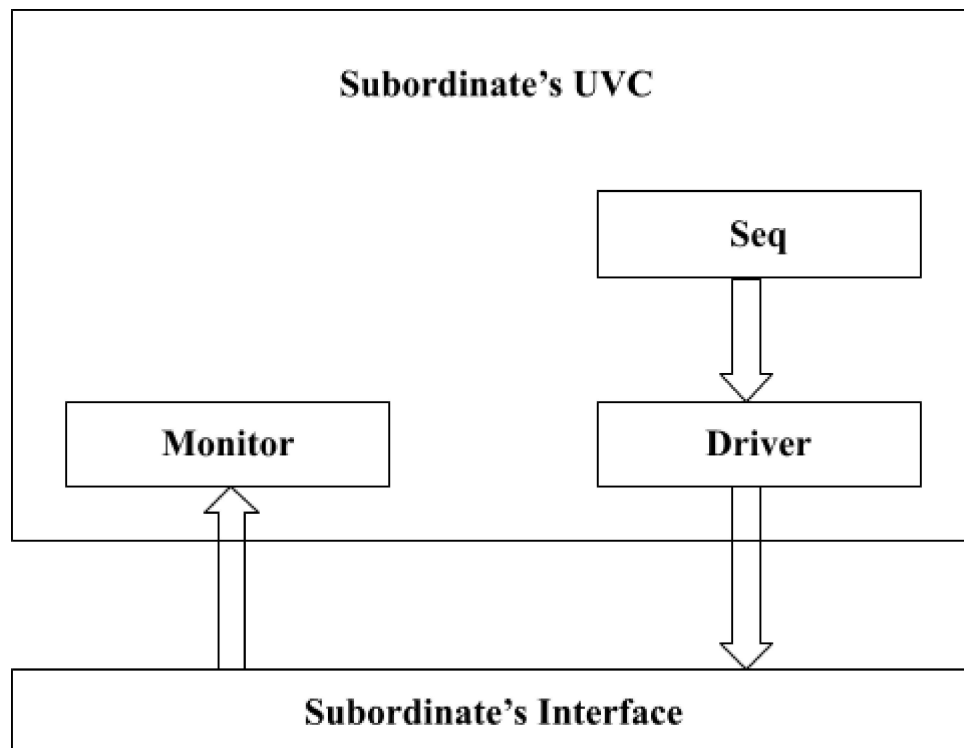
The Sequencer is connected to the Driver and provides sequences to the Driver. These sequences are used to generate the transactions that the Subordinate will respond to. The Driver is connected to the Subordinate's interface and drives the sequences onto the interface, simulating the behavior of the Subordinate.

The Monitor is connected to the Subordinate's interface and receives the sequences from the interface. It monitors the signals on the Subordinate's side to ensure that the interface works as expected.

The verification of the Subordinate's side of the AHB interface is achieved by using SystemVerilog Assertions in the interface. The assertions check whether the Subordinate side of the interface behaves correctly according to

the AHB protocol rules. In addition, coverage of the Subordinate's signals is also implemented in the interface.

The assertions in the Subordinate's interface verify certain conditions. One of the checks is whether the ERROR response is a two-cycle response. Another check is to verify if READYOUT is 1 during the second cycle of an ERROR response. These checks ensure that the Subordinate's side follows the correct protocol for error handling and response timing.



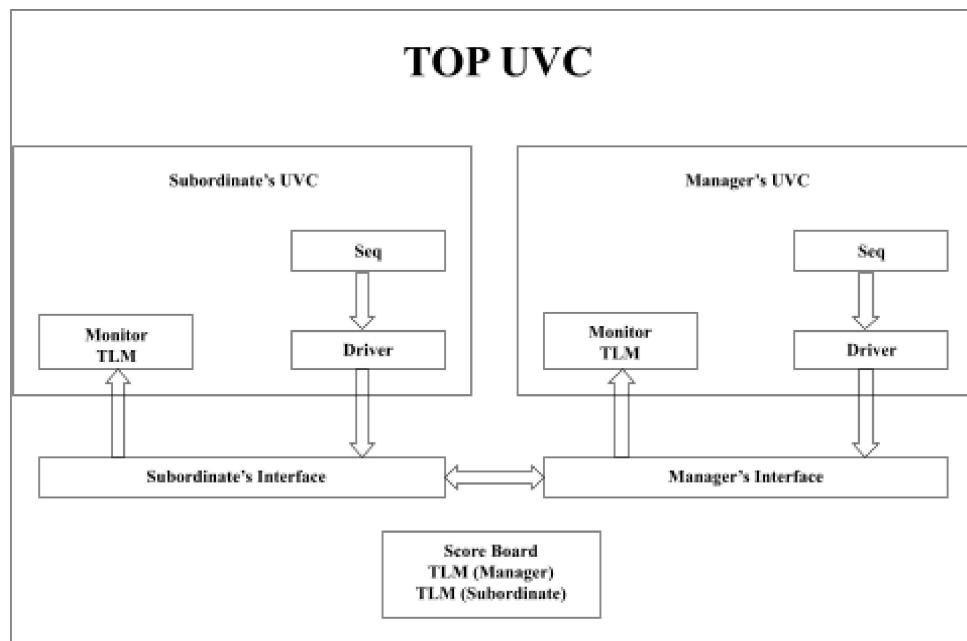
Top UVC

While the Manager's and Subordinate's UVCs individually verify the correctness of their respective interfaces using assertions, the next step is to check the communication between the Manager and Subordinate. The goal is to verify whether the data transfer between the two is successful, meaning

that the correct data is received at the correct address without any timing constraint issues.

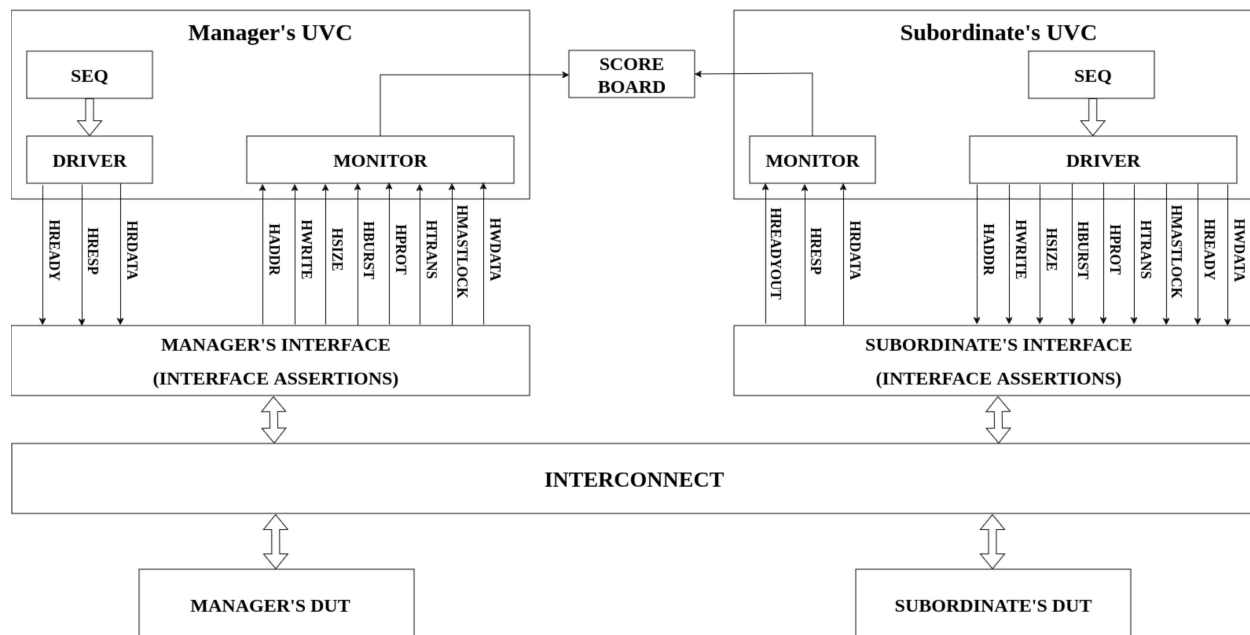
This is achieved by the Top UVC. The Top UVC contains the Scoreboard, Manager's UVC, and Subordinate's UVC. The Manager's UVC and Subordinate's UVC are connected to the Scoreboard through Transaction-Level Modeling (TLM). The Scoreboard plays a crucial role in verifying whether the communication and data transfer between the Manager and Subordinate are correctly performed.

The Scoreboard checks whether the correct data is received at the correct address, ensuring that there are no timing constraint issues during the data transfer. This process helps confirm the overall functionality and correctness of the AHB protocol in terms of communication between the Manager and Subordinate.



Below is the elaborate diagram of Top UVC. It can be observed that both the manager's and subordinate's UVCs are instantiated in the top UVC, and their monitors are connected with its scoreboard. As discussed earlier, assertions in

the interface check the control behavior of AHB, and the scoreboard verifies the correct transfer of data.

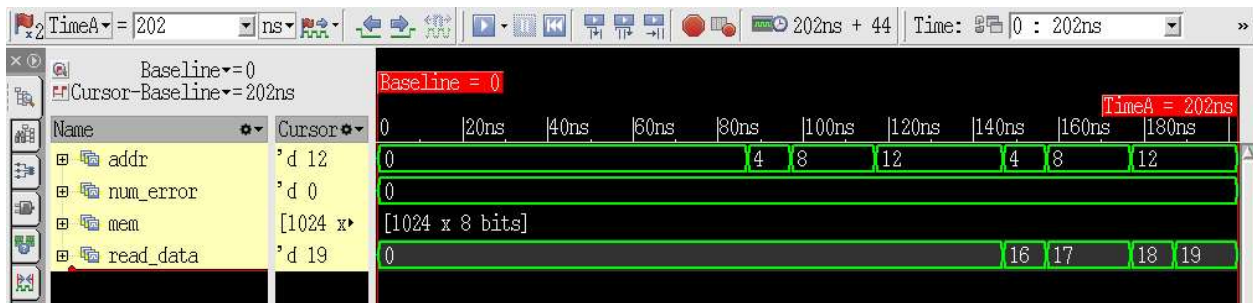


The Scoreboard of Top UVC is used to verify the communication between Manager and Subordinate, and to check whether correct data is transferred between them at the correct address. For this purpose multiple read after write sequences are implemented. The Subordinate's UVC sends 'HWDATA' to the monitor of Manager's UVC. Then Manager's UVC writes this data in TLM FIFO in the scoreboard of Top UVC. After completion of all data beats, Manager's UVC sends 'HRDATA' to the monitor of Subordinate's UVC and then Subordinate's UVC writes this data to another TLM FIFO in the scoreboard of TOP UVC. After that, the scoreboard reads data from both FIFOs and compares them to check whether correct data is transferred or not. The results of implemented tests in the scoreboard of Top UVC are attached below.

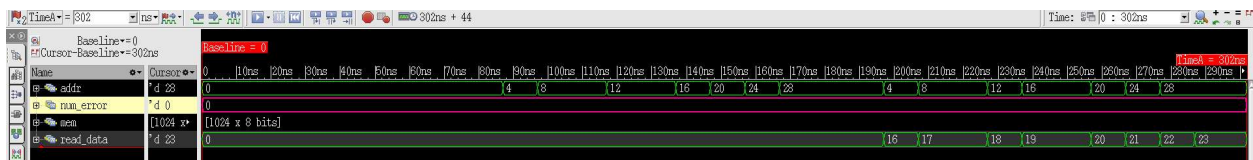
The below image is the snap of the result of a single burst read after write test.



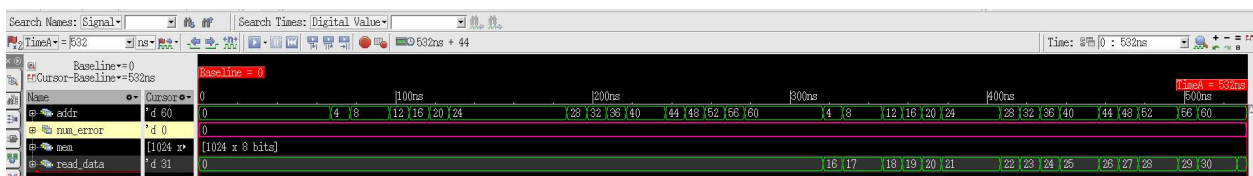
The next test is INCR4 read after write :



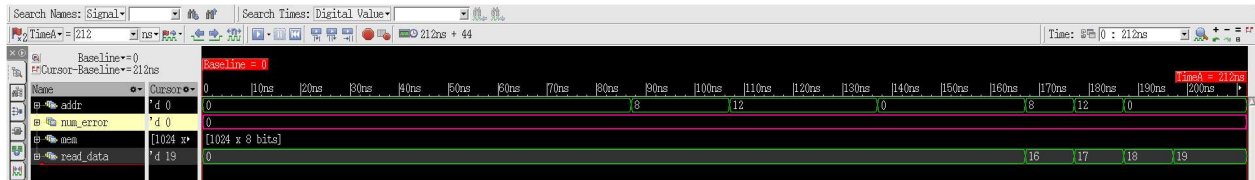
Similarly, following snap is of INCR8 read after write test :



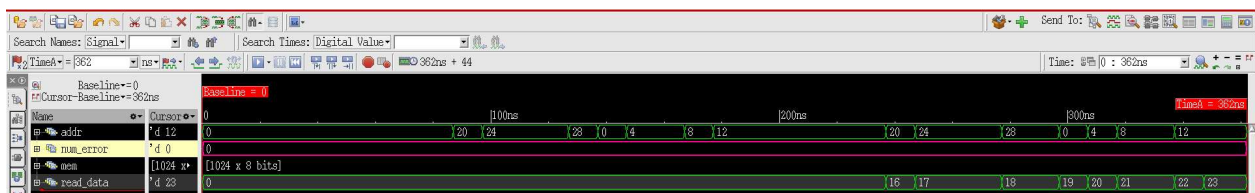
The next test is INCR16 read after write :



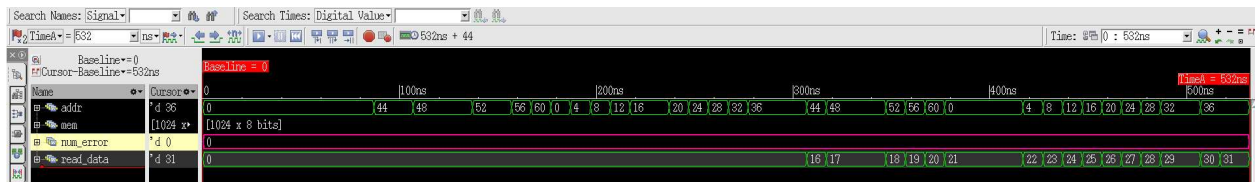
The next test is WRAP4 read after write :



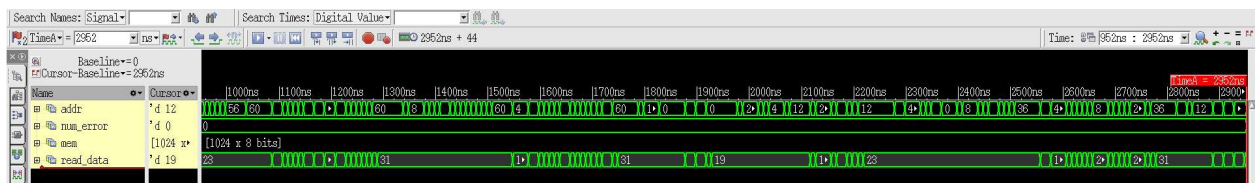
The next test is WRAP8 read after write :



The next test is WRAP16 read after write :

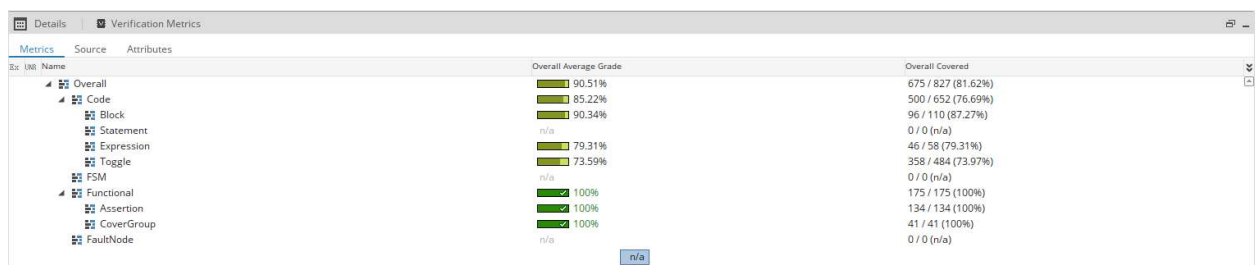


Lastly, all previous tests are run as a single test to collect coverage. Below it's snap is given :



COVERAGE

In digital design verification, **coverage** is the measurement of how much functional behaviour and features of the design under test are tested by the test bench and how much code is executed. Below is the snap, which shows the overall coverage results :



There are two types of coverage :

- **Functional Coverage** : It measures how much of design specifications or functionalities are exercised, which are captured in a test bench.
- **Code Coverage** : It measures how much of the design code is exercised by test bench.

Below is the screenshot of overall code coverage :



Similarly, Below is the snap of overall functional coverage :



Coverage of **Subordinate's UVC** is also observed. Below snap shows the coverage of data signals of Subordinate's UVC.

ahb_slave_pkg	100%	37 / 37 (100%)
ahb_slave_driver::slave_if_cov	100%	37 / 37 (100%)
HSEL	100%	1 / 1 (100%)
HWRITE	100%	2 / 2 (100%)
HADDR	100%	1 / 1 (100%)
HSIZE	100%	1 / 1 (100%)
HREADY	100%	2 / 2 (100%)
HBURST	100%	8 / 8 (100%)
HPROT	100%	1 / 1 (100%)
HTRANS	100%	4 / 4 (100%)
HMASTLOCK	100%	2 / 2 (100%)
HWDATA	100%	1 / 1 (100%)

Similarly, Cross Coverage of across the signals is :

AxB HTRANSxHREADY1	100%	3 / 3 (100%)
AxB HTRANSxHREADY2	100%	1 / 1 (100%)
AxB HBURSTxHREADY	100%	8 / 8 (100%)
AxB HMASTLOCKxHREADY	100%	2 / 2 (100%)

Subordinate's assertions are also represented and below snap shows this :

slave_vif	37.07%	86 / 120 (71.67%)
cover_error_happening_transaction_termination	0%	0 / 1 (0%)
cover_error_behaviour	0%	0 / 1 (0%)
error_happening_transaction_termination	0%	0 / 1 (0%)
error_behaviour	0%	0 / 1 (0%)

Coverage of **Manager's UVC** is also analyzed as well. Following snap shows the coverage of Manager's signals :

ahb_master_driver::ahb_slav_cov	100%	4 / 4 (100%)
vif_HREADY	100%	2 / 2 (100%)
vif_HRESP	100%	1 / 1 (100%)
vif_HRDATA	100%	1 / 1 (100%)

Coverage of Manager's assertion is :

ahb_master_if	85.98%	166 / 202 (82.18%)	100%
__cover33_WRAP16_control_signals_retention	100%	1 / 1 (100%)	n/a
__cover32_WRAP16_addr_incr_on_HReady	100%	1 / 1 (100%)	n/a
__cover31_WRAP16_addr_incr_on_BUSY	100%	1 / 1 (100%)	n/a
__cover30_WRAP16_wrap_around_check	100%	1 / 1 (100%)	n/a
__cover29_WRAP16_start	100%	1 / 1 (100%)	n/a
__cover28_WRAP8_control_signals_retention	100%	1 / 1 (100%)	n/a
__cover27_WRAP8_addr_incr_on_HReady	100%	1 / 1 (100%)	n/a
__cover26_WRAP8_addr_incr_on_BUSY	100%	1 / 1 (100%)	n/a
__cover25_WRAP8_wrap_around_check	100%	1 / 1 (100%)	n/a
__cover24_WRAP8_start	100%	1 / 1 (100%)	n/a
__cover23_WRAP4_control_signals_retention	100%	1 / 1 (100%)	n/a
__cover22_WRAP4_addr_incr_on_BUSY	100%	1 / 1 (100%)	n/a
__cover21_WRAP4_addr_incr_on_HReady	100%	1 / 1 (100%)	n/a
__cover20_WRAP4_wrap_around_check	100%	1 / 1 (100%)	n/a
__cover19_WRAP4_start	100%	1 / 1 (100%)	n/a
__cover18_locked_transfer_1	100%	1 / 1 (100%)	n/a
__cover17_locked_transfer_2	100%	1 / 1 (100%)	n/a
__cover16_undef_INCR_control_signals_retention	100%	1 / 1 (100%)	n/a
__cover15_undef_INCR_addr_incr_on_BUSY	100%	1 / 1 (100%)	n/a
__cover14_undef_INCR_addr_incr_on_HReady	100%	1 / 1 (100%)	n/a
__cover13_undef_INCR_start	100%	1 / 1 (100%)	n/a
__cover12_INCR16_control_signals_retention	100%	1 / 1 (100%)	n/a
__cover11_INCR16_addr_incr_on_BUSY	100%	1 / 1 (100%)	n/a
__cover10_INCR16_addr_incr_on_HReady	100%	1 / 1 (100%)	n/a
__cover9_INCR16_start	100%	1 / 1 (100%)	n/a
__cover8_INCR8_control_signals_retention	100%	1 / 1 (100%)	n/a
__cover7_INCR8_addr_incr_on_BUSY	100%	1 / 1 (100%)	n/a
__cover6_INCR8_addr_incr_on_HReady	100%	1 / 1 (100%)	n/a
__cover5_INCR8_start	100%	1 / 1 (100%)	n/a
__cover4_INCR4_control_signals_retention	100%	1 / 1 (100%)	n/a
__cover3_INCR4_addr_incr_on_BUSY	100%	1 / 1 (100%)	n/a
__cover2_INCR4_addr_incr_on_HReady	100%	1 / 1 (100%)	n/a
__cover1_INCR4_start	100%	1 / 1 (100%)	n/a
single_burst_read_after_write	100%	1 / 1 (100%)	100%
INCR4_start	100%	1 / 1 (100%)	100%
INCR4_addr_incr_on_HReady	100%	1 / 1 (100%)	100%
INCR4_addr_incr_on_BUSY	100%	1 / 1 (100%)	100%
INCR4_control_signals_retention	100%	1 / 1 (100%)	100%
INCR8_start	100%	1 / 1 (100%)	100%

None			
WRAP8_wrap_around_check	100%	1 / 1 (100%)	100%
WRAP8_addr_incr_on_HReady	100%	1 / 1 (100%)	100%
WRAP8_addr_incr_on_BUSY	100%	1 / 1 (100%)	100%
WRAP8_control_signals_retention	100%	1 / 1 (100%)	100%
WRAP16_start	100%	1 / 1 (100%)	100%
WRAP16_wrap_around_check	100%	1 / 1 (100%)	100%
WRAP16_addr_incr_on_HReady	100%	1 / 1 (100%)	100%
WRAP16_addr_incr_on_BUSY	100%	1 / 1 (100%)	100%
WRAP16_control_signals_retention	100%	1 / 1 (100%)	100%
undef_INCR_start	100%	1 / 1 (100%)	100%
undef_INCR_addr_incr_on_HReady	100%	1 / 1 (100%)	100%
undef_INCR_addr_incr_on_BUSY	100%	1 / 1 (100%)	100%
undef_INCR_control_signals_retention	100%	1 / 1 (100%)	100%
locked_transfer_2	100%	1 / 1 (100%)	100%
locked_transfer_1	100%	1 / 1 (100%)	100%